Custom-Object Character Recognition (OCR) on AWS (Google Drive / Cloud Storage)

Harleen Kaur

October 10, 2026

Abstract

This project builds a custom OCR system that reads the key fields of a Thyrocare lab report and saves them as an editable CSV and Excel table. A YOLOv3 model finds five field types on the report image; each detected region is cropped, cleaned with OpenCV and read by Tesseract.

On 10 unseen test images the detector reached **mAP50 0.975** (precision 0.968, recall 0.956), and the final OCR text matched the hand-checked answers with an overall similarity of **0.938**. The whole project ran on Google Colab (free T4 GPU) and Google Drive, so the cost was **$0**.

1. Objective and pipeline

The objective is to read specific fields of a lab report automatically and turn them into an editable file. The brief asks for a custom OCR that combines YOLO (to find the fields) with Tesseract (to read them), trained on a personal dataset.

The system looks for five field types on every report: `patient_info`, `test_name`, `value`, `units` and `ref_range`. Each report goes through six stages:

1. **Detect.** The trained YOLOv3 model finds the five field types and returns a bounding box for each.
2. **Crop.** Each box is cut out of the original full-size photo and stored in a list.
3. **Preprocess.** Each crop is enlarged 3x, converted to grayscale, lightly blurred (Gaussian), thresholded with Otsu's method (white text on black), then flipped with `bitwise_not` to black text on white, which Tesseract reads best.
4. **Read.** Each cleaned crop is passed to Tesseract, with stricter settings for numbers and units.
5. **Clean up.** Duplicate boxes are removed, and units and test names are matched to their correct spellings.
6. **Save.** The result is written to CSV and Excel files in Google Drive.

The command from the brief runs the whole pipeline: `python Custom_OCR.py --image <yourimage.jpg>`.

2. Environment and tools

Everything ran on free tools: Google Colab for compute, Google Drive for storage and Roboflow for labelling. The brief's cost section describes this Colab and Drive setup, so it was used in place of AWS SageMaker and S3.

Component	What was used	Role
Compute	Google Colab, Tesla T4 GPU, Python 3.13	Training and inference
Storage	Google Drive (15 GB free)	Dataset, weights and results
Detector	YOLOv3 through Ultralytics 8.4.175 (`yolov3u`, 103.7 million parameters)	Finds the five field types
OCR engine	Tesseract with pytesseract	Reads text from each crop
Image processing	OpenCV	Crop, resize, grayscale, blur, threshold
Labelling	Roboflow (free plan)	Drawing and exporting the YOLO labels

The Drive folder `Custom_OCR` holds three folders:

- `datasets/` for the raw images, the Roboflow export and the cleaned labels
- `models/` for the trained weights (`yolov3_ocr/weights/best.pt`)
- `results/` for the OCR outputs, the cropped images and the evaluation files

Estimated cost: $0. Colab's free tier, the first 15 GB of Drive and Roboflow's free plan were enough for 100 images.

3. Task 1: Environment setup

The workspace was a new Colab notebook with the T4 GPU enabled (Runtime, Change runtime type, T4 GPU). Google Drive was mounted so that the dataset, weights and results survive after the session ends, and the folder structure from section 2 was created.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
import os
base = '/content/drive/MyDrive/Custom_OCR'
for folder in ['datasets', 'models', 'results']:
    os.makedirs(f'{base}/{folder}', exist_ok=True)
```

Tesseract, pytesseract and OpenCV were then installed, and a check confirmed that OpenCV, Tesseract and the GPU were all available.

```
!apt-get install -y tesseract-ocr
!pip install pytesseract opencv-python
```

The first mount attempt failed with `ValueError: mount failed`, as the screenshot shows.

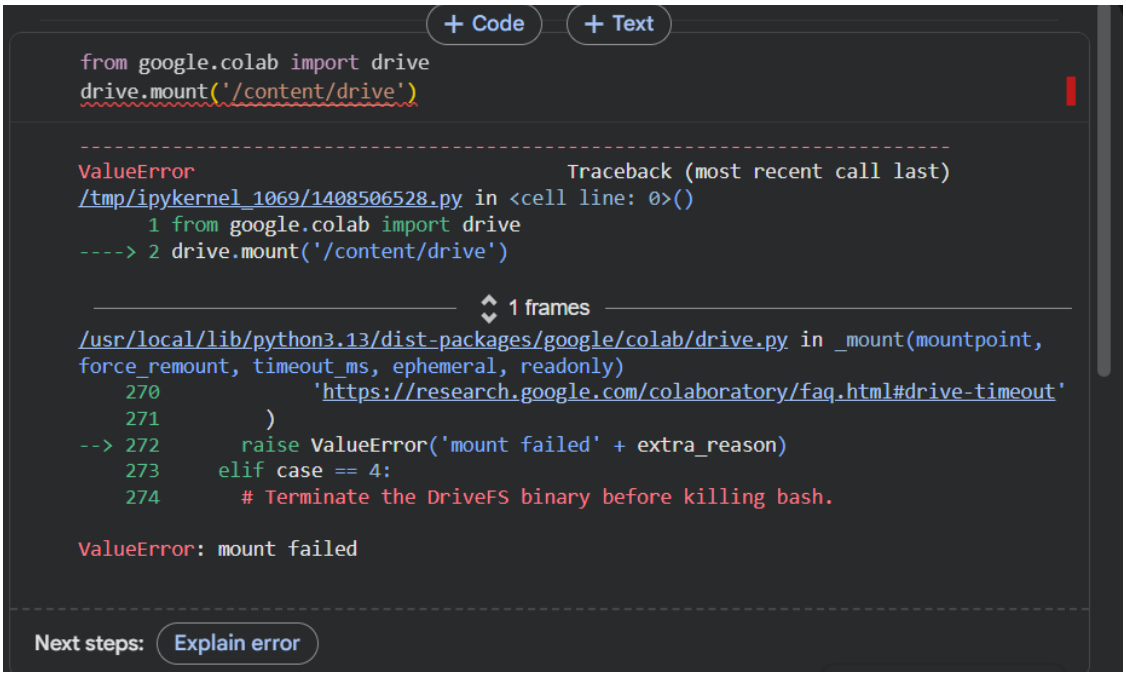


Figure 1. Drive mount error in Colab

The fix was to choose Runtime, Disconnect and delete runtime, run `drive.mount('/content/drive', force_remount=True)` again, allow the browser pop-up, and tick every permission box on the Google consent screen. The mount then worked.

4. Task 2: Data preparation

The dataset is 100 photographs of Thyrocare lab reports (JPG files of about 939 x 631, 1068 x 635 or 635 x 1043 pixels). It came without any label files, and YOLO cannot train without them, so every field was labelled by hand.

Classes. Five classes were defined:

ID	Class	What is boxed
0	`patient_info`	The block with NAME, REF. BY and TEST ASKED
1	`ref_range`	The reference or normal range text
2	`test_name`	The name of each test
3	`units`	The unit of each result (ng/ml, mg/dl and so on)
4	`value`	The numeric result

Labelling. The images were uploaded to Roboflow (free plan) and every box was drawn in its labelling tool.

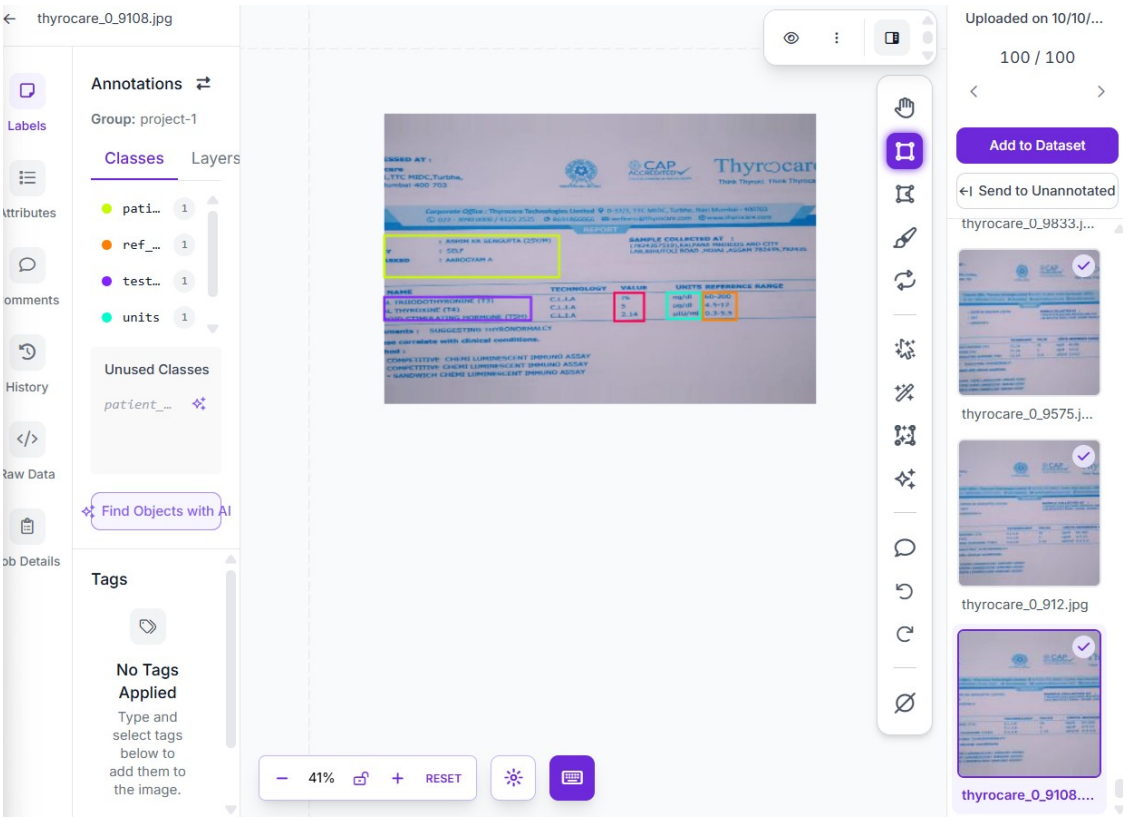


Figure 2. Labelling an image in Roboflow

The reports follow three main layouts, and each was labelled in a fixed way so that the labels stay consistent:

- **Table layout** (liver panel, 11 tests): one tall box per column covering all of its rows, plus one `patient_info` box.
- **List layout** (Vitamin D and B-12): a separate box for each test name, value, unit and reference range, because each test sits apart from the next.
- **Thyroid panel** (T3, T4, TSH): labelled like the table layout.

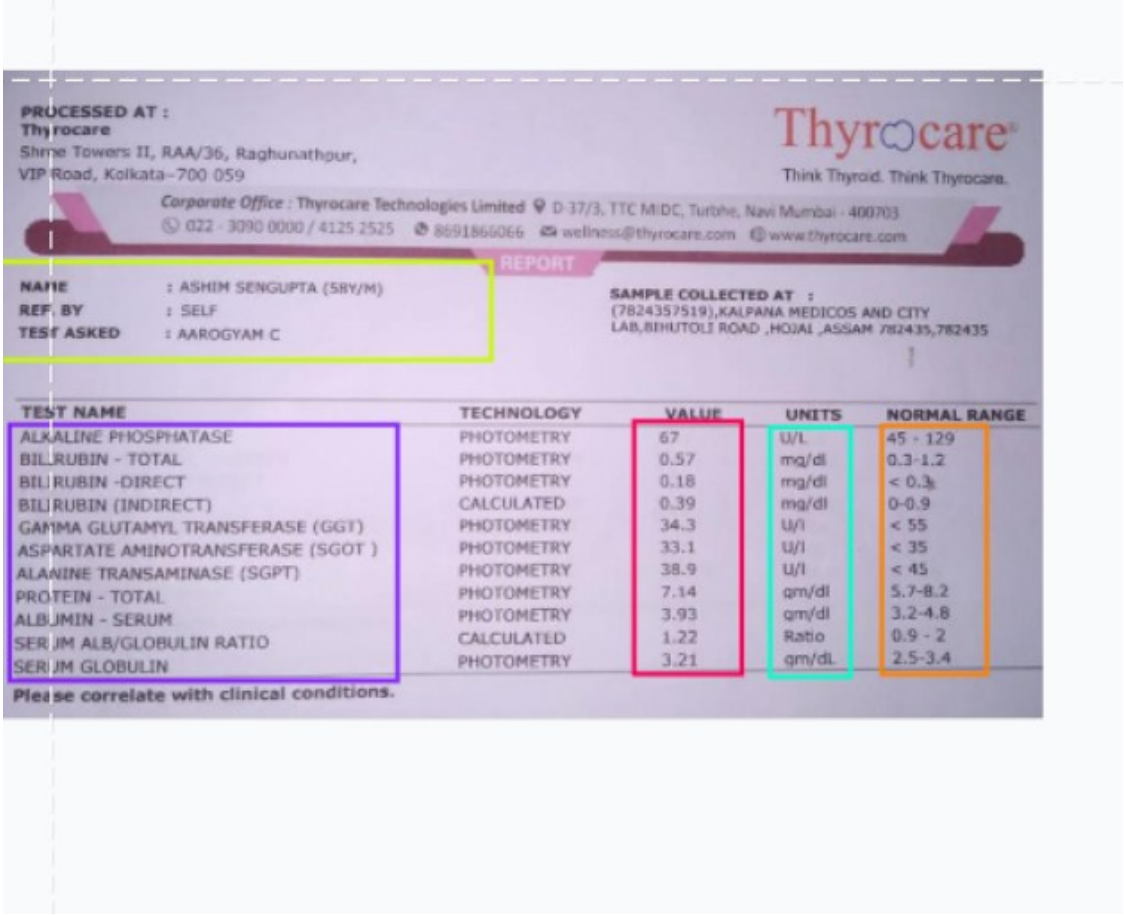


Figure 3. Table layout labelled with one box per column

The image below shows how the list layout was labelled (9 boxes on this report).

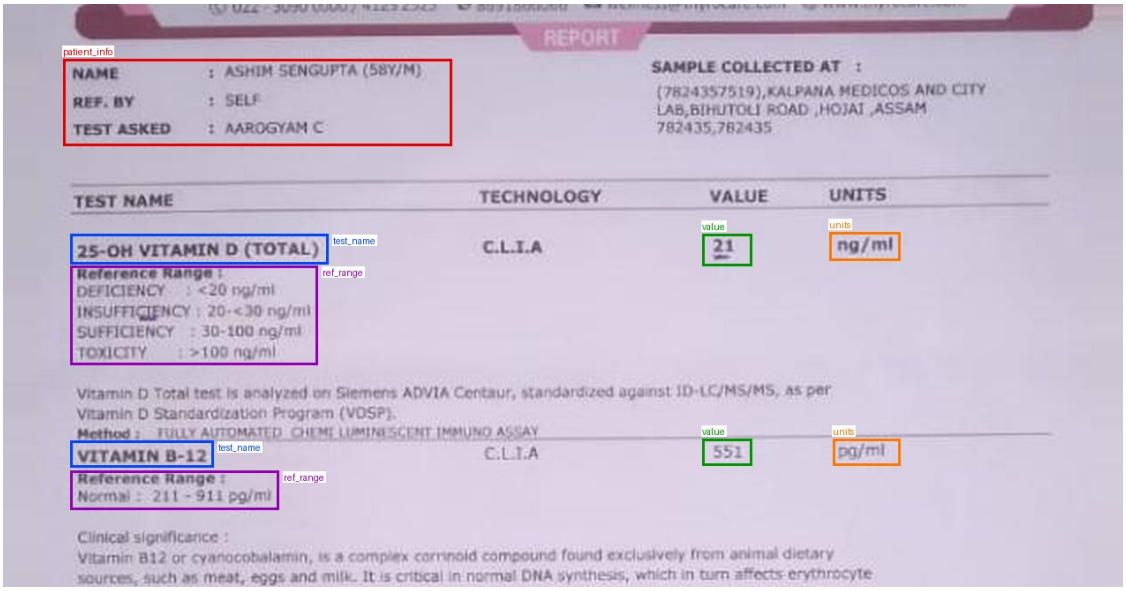


Figure 4. List layout labelled with separate boxes per test

Dataset version. All 100 images were added to the dataset, as the Roboflow board shows.

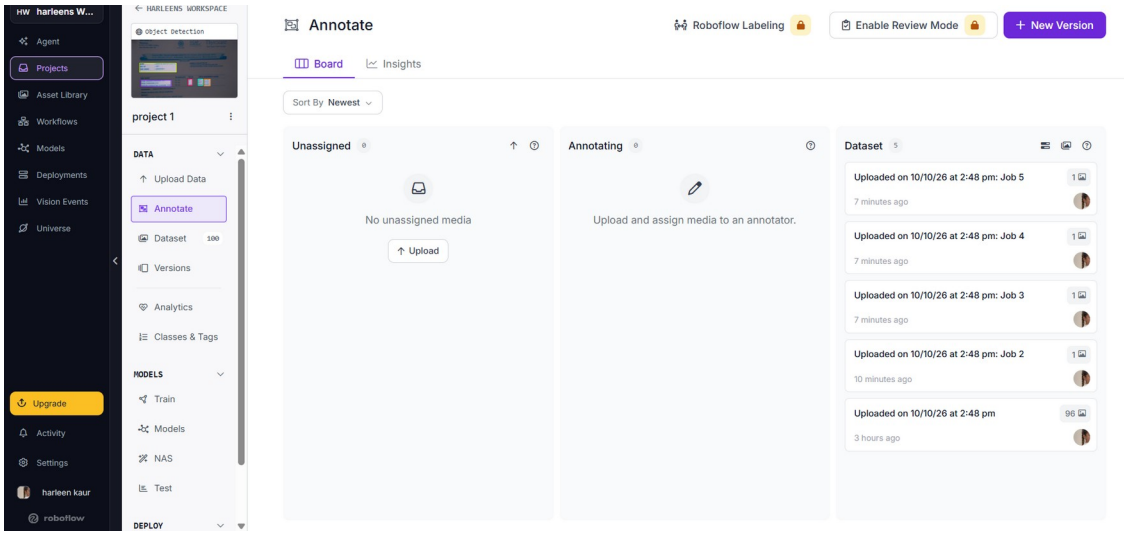


Figure 5. Roboflow board with 100 images in the dataset

The version used a 70 / 20 / 10 split into train, validation and test, a stretch-resize to 416 x 416, and no augmentation. Flip and rotate were left out on purpose, because they would mirror or tilt the text. The dataset was exported in YOLO Darknet format and unzipped in Colab. The counts matched for every split (70 train, 20 valid, 10 test images, each with its own label file).

```
train 70 images, 70 labels
valid 20 images, 20 labels
test 10 images, 10 labels
patient_info
patient_name
ref_range
test_name
units
value
```

Figure 6. Split counts and class names after unzipping

The class list printed six names instead of five. A stray class, `patient_name`, held 3 boxes that belong to `patient_info`. A short script merged them and renumbered the classes, and the clean copy was saved as `datasets/yolo_clean`. It has 101 `patient_info`, 127 `ref_range`, 129 `test_name`, 128 `units` and 127 `value` boxes.

5. Task 3: Model training

The detector is YOLOv3 (96 fused layers, 103.7 million parameters), started from the pretrained `yolov3u.pt` weights through the Ultralytics library. The labelled images were copied to Colab's local

disk in the `images/` and `labels/` folder layout YOLO expects, with a `data.yaml` file listing the five classes.

Setting	Value	Reason
Epochs	up to 100, stop early after 30 without improvement	Enough for 70 training images
Image size	416	Matches the 416 x 416 export
Batch size	8	Fits the T4 GPU memory
Horizontal and vertical flip	off	Mirrored text cannot be read
Output folder	`Custom_OCR/models/yolov3_ocr` in Drive	Weights survive the session

Training ran on the Colab T4 GPU. The log below is from epoch 29 of 100: on the 20 validation images (132 boxes) recall was rising from epoch to epoch and the losses were still falling.

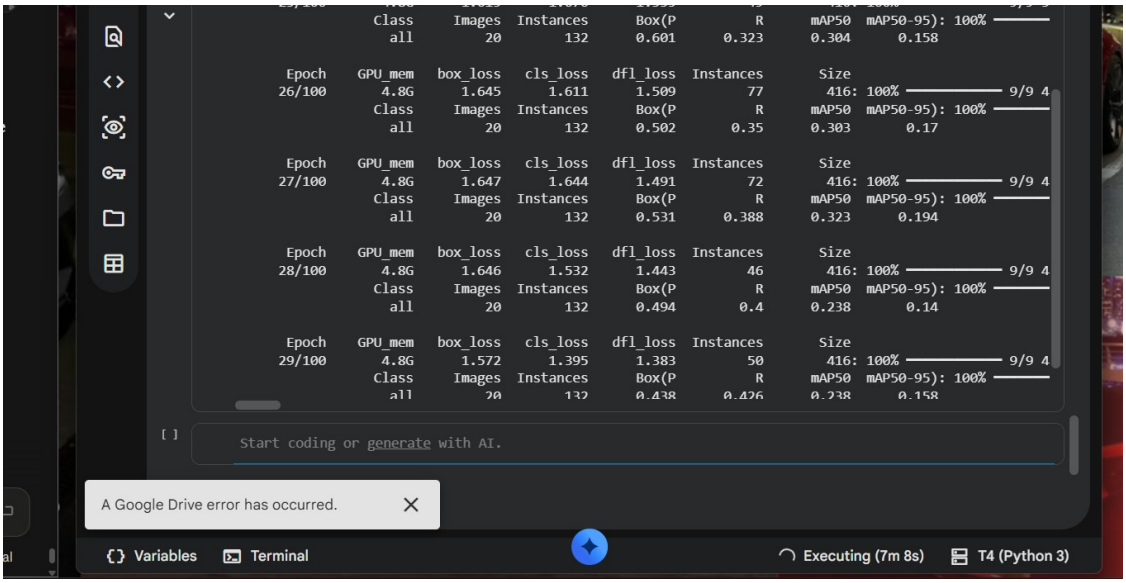


Figure 7. Training log at epoch 29 of 100

Training saved `best.pt` and `last.pt` (about 199 MB each) in `models/yolov3_ocr/weights/` in Drive. `best.pt` is the final model.

Test-set results. The model was then scored on the 10 test images, which it had never seen (62 boxes in total):

Measure	Result
Precision	0.968
Recall	0.956
mAP50	0.975
mAP50-95	0.612

Per class, mAP50 was 0.995 for `patient_info`, `ref_range` and `units`, 0.966 for `test_name` and 0.925 for `value`. The weakest class is `value` probably because the numbers are small and close together.

6. Task 4: Inference and OCR

For each report, the image is stretched to 416 x 416 (the same shape used in training) and passed to the model. The returned boxes are scaled back to the original full-size photo, and each region is cropped with a few pixels of margin so that no letters are cut off. Cropping from the original keeps the text sharp for Tesseract.

Each crop then goes through the preprocessing steps from the brief:

```
def preprocess(crop):
    crop = cv2.resize(crop, None, fx=3, fy=3,
interpolation=cv2.INTER_CUBIC)   # blow the crop up 3x
    gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
# grayscale
    blur = cv2.GaussianBlur(gray, (3, 3), 0)
# small blur to smooth noise
    thresh = cv2.threshold(blur, 0, 255,
                           cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)[1]
# Otsu: white text on black
    return cv2.bitwise_not(thresh)
# flip to black text on white
```

The cleaned crop is read with `pytesseract.image_to_string` and every line of text is joined with `|`. The first run on one report produced a table of the detected fields:

Saved /content/drive/MyDrive/Custom_OCR/results/one_image.csv

	class	conf	text
0	patient_info	0.81	WAMe 1 ASHIM SENGUPTA (58Y/M} | ReP. BY + SEU | TEST ASKED 3 AAROGYAM C
1	ref_range	0.97	Reference Range : | OBFICIENCY = <20 ng/mi | INSUFFICIENCY ; 20-< 20 ng/m! |...
2	ref_range	0.86	Reference Range: | Normal: 211-911 pgymi
3	test_name	0.66	25-08 VITAMIN D (TOTAL)
4	test_name	0.28	25-08 VITAMIN D (TOTAL)
5	test_name	0.28	Method: buuly Aun | VITAMIN B-12
6	test_name	0.31	VITAMIN B-12
7	units	0.53	ng/ml
8	units	0.56	pg/mi
9	value	0.44	21
10	value	0.39	351

Figure 8. First OCR output on one report

All five field types were found and most of the text was right (for example `25-OH VITAMIN D (TOTAL)`, `ng/ml` and `21`). The run also showed what needed fixing: the same test was detected twice, `551` was read as `351`, `pg/ml` as `pg/mi`, and one test-name box picked up extra text from the line above it.

7. Task 5: Evaluation and optimization

OCR quality was measured against a ground truth: the correct text of every field on the 10 test images (10 images x 5 fields = 50 answers), written by reading the original reports. One image, `thyrocare_0_9833`, is cut off at the left edge, so its answer contains only the visible text.

Two scores were used. **Similarity** is the share of characters that match (0 to 1, from Python's `difflib`). **Exact** is the share of fields that match perfectly. Exact is very strict, because one wrong character in a block of several lines counts as a miss, so similarity is the fairer number.

```
             similarity  exact
class
patient_info       0.932    0.1
ref_range          0.925    0.5
test_name          0.895    0.2
units              0.808    0.1
value              0.950    0.8
Overall similarity: 0.902
```

Figure 9. Scores of the first run, before any cleanup

Four changes were made to improve the first run:

1. **Duplicate boxes removed.** Two boxes of the same class that overlap by more than 40% are merged, keeping the more confident one.
2. **Stricter Tesseract settings per field.** `value` may contain only digits and `. < > -`; `units` only letters, digits and `/ % . ^ µ`.
3. **Wider crop margin** of 4 pixels, so that edge letters are not lost.
4. **A cleanup stage.** Each unit and test name is matched to the nearest known spelling (for example `ng/di` becomes `ng/dl` and `25-08 VITAMIN D` becomes `25-OH VITAMIN D`), junk lines such as `Method: buuly Aun` are dropped, and a test name read twice is kept once.

The table shows the similarity score after each stage:

Field	First run	After cleanup	Final script
`patient_info`	0.932	0.932	0.932
`ref_range`	0.925	0.925	0.925
`test_name`	0.895	0.980	0.994
`units`	0.808	0.888	0.888
`value`	0.950	0.950	0.950
Overall	**0.902**	**0.935**	**0.938**

The exact-match rate for `test_name` rose from 0.2 to 0.8, for `ref_range` from 0.1 to 0.5 and for `units` from 0.1 to 0.3 after the cleanup. The improved output of the same report as before is below.

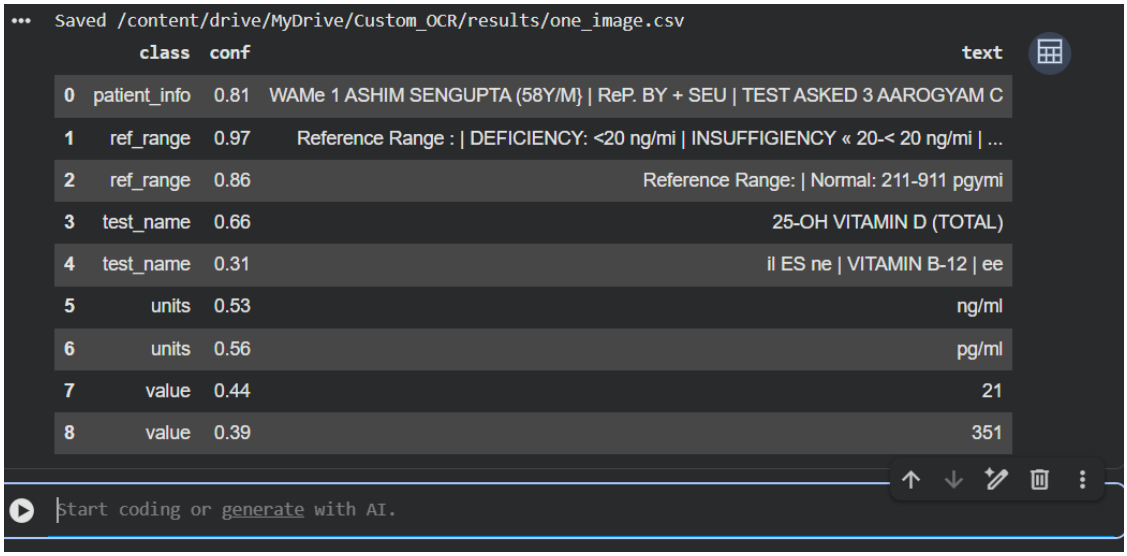


Figure 10. OCR output after the improvements

8. Task 6: Automation

All stages are combined in one script, `Custom_OCR.py`, saved in the `Custom_OCR` folder in Drive. One command processes a single image or a whole folder:

```
# one report
python Custom_OCR.py --image thyrocare_0_8491.jpg --out results/demo

# every .jpg in a folder
python Custom_OCR.py --folder /content/test_orig --out results
```

The script writes three files to the output folder, plus the cleaned crops in `crops/`:

File	Content
`ocr_output.csv`	One row per detected field: image, class, box coordinates, confidence, text
`report.csv`	The editable table: one row per test with patient info, test name, value, units and reference range
`report.xlsx`	The same table as an Excel file

The run on the demo report, and the report table it produced, are shown below.

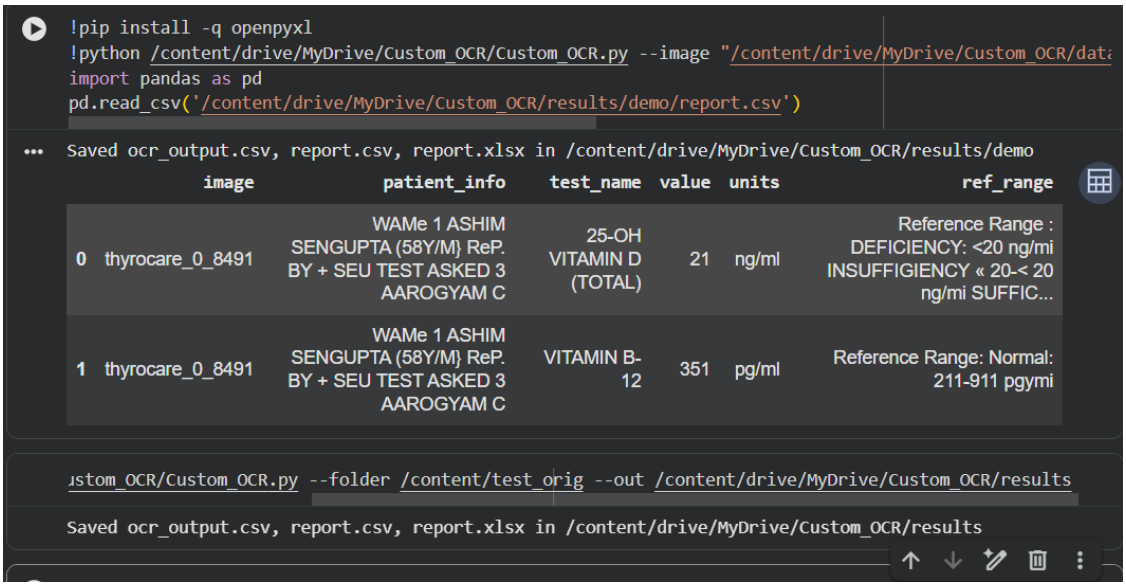


Figure 11. Script run and the resulting report table

The table has the structure of the original report: **25-OH VITAMIN D (TOTAL), 21, ng/ml** and **VITAMIN B-12, pg/ml**, each with its reference range. The value 351 for Vitamin B-12 should read 551; this is the small-digit error described in the limitations.

9. Results summary

The model finds the fields almost every time, and the text it reads is about 94% correct on the unseen test images.

Item	Result
Dataset	100 labelled reports, 5 classes (70 train, 20 validation, 10 test)
Detector	YOLOv3, mAP50 0.975, precision 0.968, recall 0.956 on the test set
Best detected fields	`patient_info`, `ref_range`, `units` (mAP50 0.995)
Weakest detected field	`value` (mAP50 0.925)
OCR accuracy	Overall similarity 0.938 (first run 0.902)
Best read field	`test_name` (0.994)
Weakest read field	`units` (0.888)
Outputs	`ocr_output.csv`, `report.csv`, `report.xlsx`
Cost	$0

10. Problems faced and how they were solved

Problem	Cause	Fix
`ValueError: mount failed` when mounting Drive	Permission step blocked or timed out	Delete the runtime, remount with `force_remount=True`, allow pop-ups and tick all permissions

Problem	Cause	Fix
No labels in the dataset	The 100 images came without annotations	Labelled every field by hand in Roboflow
Six classes instead of five	A stray `patient_name` class held 3 boxes	Merged it into `patient_info` and renumbered the classes
"A Google Drive error has occurred" during training	A brief Colab-to-Drive connection message; training was not interrupted	Training continued; `best.pt` and `last.pt` were checked in Drive afterwards
`ModuleNotFoundError: No module named 'pytesseract'`	Colab forgets installed libraries when the runtime restarts	Reinstalled it; the install cell is now run first each session
The same test detected twice	Overlapping boxes of one class	Overlap-based duplicate removal
`ng/di`, `pg/mi`, `25-08 VITAMIN D`	Tesseract confuses `l`, `i`, `!` and `0`, `H`	Per-field character lists and matching to known spellings

The screenshot shows the missing-module error and its fix.

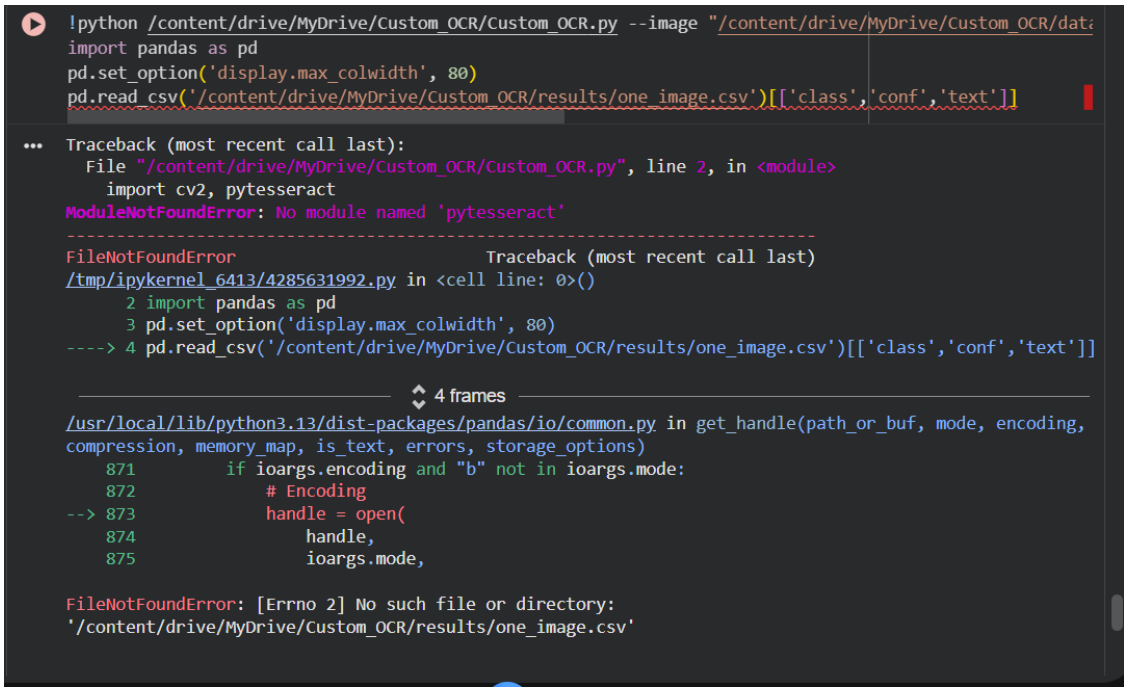


Figure 12. pytesseract missing in a fresh Colab session

11. Limitations and future work

Limitations

- The test set has only 10 images from one lab and a few layouts, so the scores may be lower on reports from other labs.
- The cleanup stage uses a list of known units and test names. A test name that is not on the list keeps whatever OCR errors Tesseract made.

- Small digits are sometimes misread (`551` read as `351`), which is the main remaining error in `value`.
- `patient_info` has noise such as `NAME ;` instead of `NAME :`, so it rarely matches exactly even though the similarity is 0.93.
- The answers used for scoring were written by reading the images; one image is partly cut off.

Future work

- Label more reports, including other layouts, and add light augmentation such as brightness and blur changes.
- Train at a larger image size (for example 640) so small text is detected more reliably.
- Compare Tesseract with another OCR engine for the digits.
- Add a step that parses `patient_info` into separate name, age and test fields.
- Extend the list of known test names and units as more reports are added.
- Wrap the script in a small web page so a report can be uploaded and the Excel file downloaded.

12. How to run the project, and conclusion

To run the OCR on a new report in a fresh Colab session:

1. Open a Colab notebook and set the runtime to T4 GPU.
2. Mount Google Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

3. Install the libraries (this has to be repeated in every new session):

```
!apt-get -qq install tesseract-ocr
!pip install -q ultralytics pytesseract openpyxl
```

4. Run the script on an image, as in the brief:

```
!python /content/drive/MyDrive/Custom_OCR/Custom_OCR.py --image
<yourimage.jpg> --out /content/drive/MyDrive/Custom_OCR/results
```

The files saved in Google Drive under `Custom_OCR`:

Path	Content
`datasets/images_raw/`	The 100 original report images
`datasets/yolo_data/` and `datasets/yolo_clean/`	The Roboflow export and the cleaned five-class labels
`models/yolov3_ocr/weights/best.pt`	The trained YOLOv3 weights
`Custom_OCR.py`	The complete pipeline script
`results/ground_truth_final`	The correct answers for the 10 test images

Path	Content
`.csv`	
`results/ocr_output.csv`, `report.csv`, `report.xlsx`	OCR results on the test images
`results/crops/`	The cleaned crop of every detected field

Conclusion. Combining a YOLOv3 detector with Tesseract turns a lab-report photograph into an editable table. The detector reached 0.975 mAP50, the OCR text matched the correct answers with 0.938 similarity, and the whole project cost nothing to run. More labelled layouts and a better digit reader are the next steps to make it work on reports from other labs.